

Guia de plataformas online

Onde rodar, sem instalar nada, todos os códigos da disciplina

Disciplina	Curso	Carga horária	Encontros
Paradigmas de Programação	Ciência da Computação / Sistemas de Informação	60h	20 de 3 horas

Para que serve este guia

A disciplina usa sete linguagens como instrumentos de investigação dos conceitos: Python, Racket, Haskell, Elixir, Go, SWI-Prolog e JavaScript, além de Java nos comparativos de tipagem e de concorrência. Instalar todas em uma máquina de laboratório leva mais tempo do que a aula tem. Este guia lista, para cada linguagem, uma plataforma que roda o código direto no navegador, sem instalação e sem cadastro obrigatório — e diz o que cada uma **não** consegue fazer, porque toda plataforma online tem limites e é melhor conhecê-los antes do que descobri-los no meio do laboratório.

Todos os oito arquivos au1a01.* distribuídos com este guia foram executados e conferidos: as oito versões resolvem o mesmo problema e imprimem o mesmo resultado, 1.333.353.333.400.000. Se a sua saída terminar com >>> AMBIENTE OK <<<, o ambiente está pronto.

A regra dos três níveis

1. Playground oficial da linguagem — sempre a primeira escolha: versão atual, mantido pela própria comunidade, sem cadastro. **2. Compilador genérico multi-linguagem** — quando não há playground oficial, ou quando ele está fora do ar. **3. Instalação local** — obrigatória para os trabalhos maiores: projeto do interpretador, sistema concorrente em Elixir e Go, e experimentos com JAX.

Tabela mestre

Uma linha por linguagem. A coluna “Cadastro” indica se é preciso criar conta para simplesmente executar um programa.

Linguagem	Plataforma recomendada	Endereço	Cadastro	Aulas
Python 3	Google Colab	colab.research.google.com	conta Google	1–8, 11–16
Racket	OneCompiler · Racket	onecompiler.com/racket	não	1, 3, 6, 16
Haskell	Haskell Playground (oficial)	play.haskell.org	não	1, 4, 5, 8
Elixir	glot.io · Elixir	glot.io/new/elixir	não	1, 5, 15
Go	Go Playground (oficial)	go.dev/play	não	1, 15
SWI-Prolog	SWISH (oficial)	swish.swi-prolog.org	não	1, 12, 13
JavaScript	Console do navegador (F12)	qualquer aba, tecla F12	não	1, 8, 11
Java 17+	OneCompiler · Java	onecompiler.com/java	não	1, 8, 11, 14

Uma página por linguagem

Como rodar o arquivo da Aula 1 em cada plataforma, e o que esperar.

Python 3 — Google Colab

Arquivo: `aula01.py` · Endereço: `colab.research.google.com`

Passo a passo

1. Abra o Colab e crie um notebook novo (Arquivo → Novo notebook).
2. Cole o conteúdo de `aula01.py` em uma célula de código.
3. Execute com Ctrl+Enter. Nada precisa ser instalado.

O que o programa mostra

Quatro estilos medidos: imperativo, funcional, orientado a objetos e declarativo, com gráfico de barras em ASCII de tempo e de mutações de estado.

Limitações que você precisa conhecer

A sessão do Colab é descartada após cerca de 12 horas ou 90 minutos de inatividade — salve o notebook no Drive. Os notebooks completos da disciplina (`aula01_codigo1/2/3.ipynb`) usam `matplotlib` e produzem gráficos de verdade; o arquivo `.py` aqui usa ASCII de propósito, para rodar também em compiladores genéricos.

Alternativas sem conta: OnlineGDB, Programiz, `paiza.io`, `glot.io`.

Racket — OneCompiler

Arquivo: `aula01.rkt` · Endereço: `onecompiler.com/racket`

Passo a passo

1. Abra o endereço; o editor já vem com um exemplo.
2. Apague tudo e cole o conteúdo de `aula01.rkt`, inclusive a primeira linha `#lang racket`.
3. Clique em Run.

O que o programa mostra

Quatro versões: funcional com `foldl`, recursiva de cauda, imperativa com `set!` e um objeto construído apenas com `closure` — a prévia da Aula 11.

Limitações que você precisa conhecer

Racket não tem um playground oficial mantido pela comunidade. O antigo `try-racket.org` é um projeto pessoal de 2014 e pode estar fora do ar; não dependa dele para a aula. O OneCompiler executa Racket 8.10, suficiente para tudo que faremos. Bibliotecas gráficas (`racket/gui`, `plot`) não funcionam online.

Alternativas: JDoodle (`jdoodle.com/execute-racket-online`) e, para as Aulas 6 e 16, o DrRacket instalado localmente — que é o recomendado para o projeto do interpretador.

Haskell — Haskell Playground

Arquivo: aula01.hs · Endereço: play.haskell.org

Passo a passo

1. Abra o endereço; é o playground oficial, mantido pelo comitê do Haskell.org.
2. Cole o conteúdo de aula01.hs (o arquivo já traz module Main e main).
3. Escolha a versão do GHC no seletor e clique em Run.

O que o programa mostra

Quatro formas de escrever a mesma função — composição, compreensão, recursão com guardas e pointfree — e um gráfico de linhas de código por versão.

Limitações que você precisa conhecer

Programas de um arquivo só. O botão Core mostra a representação intermediária do GHC, um ótimo recurso para a Aula 8. Os trechos salvos geram um link compartilhável, mas o próprio playground avisa que não é sistema de armazenamento: guarde o arquivo você mesmo.

Alternativas: glot.io/new/haskell, OnlineGDB, Replit.

Elixir — glot.io

Arquivo: aula01.exs · Endereço: glot.io/new/elixir

Passo a passo

1. Abra o endereço e cole o conteúdo de aula01.exs.
2. Clique em Run. O arquivo define um módulo e executa no fim, então não precisa de mais nada.

O que o programa mostra

Pipeline com `|>`, compreensão, recursão com pattern matching e uma versão com quatro processos isolados que trocam mensagens — a prévia da Aula 15.

Limitações que você precisa conhecer

O Livebook, o notebook oficial do Elixir, é excelente mas exige instalação ou uma instância hospedada; para a Aula 1 fique no glot.io. Atenção ao OneCompiler: a versão de Elixir ali é antiga (1.9), o que basta para este código mas pode falhar com recursos mais recentes da linguagem.

Alternativas: OneCompiler, JDoodle, Ideone, tutorialspoint.

Go — Go Playground

Arquivo: aula01.go · Endereço: go.dev/play

Passo a passo

1. Abra o endereço, apague o exemplo e cole o conteúdo de aula01.go.
2. Clique em Run. Goroutines, canais e sync.Mutex funcionam normalmente.

O que o programa mostra

Imperativo, ordem superior, CSP com canais e memória compartilhada com mutex — os dois modelos de concorrência da disciplina, lado a lado.

Limitações que você precisa conhecer

O sandbox usa um relógio falso, que só avança quando todas as goroutines bloqueiam. Por isso os tempos podem sair zerados: o programa detecta essa situação e avisa, em vez de imprimir um gráfico sem sentido. Para medir tempo de verdade, rode localmente. Também não há acesso à rede nem ao sistema de arquivos.

Alternativas: glo.t.io/new/go, OnlineGDB, Replit.

SWI-Prolog — SWISH

Arquivo: aula01.pl · Endereço: swish.swi-prolog.org

Passo a passo

1. Abra o endereço e escolha “Program” ao criar um novo arquivo.
2. Cole o conteúdo de aula01.pl no painel da esquerda.
3. No painel de consulta, digite main. e pressione Enter (ou Ctrl+Enter).

O que o programa mostra

Três formas de somar — recursão com corte, findall/3 e aggregate_all/3 — e a demonstração de que a mesma regra avo/2 responde a três perguntas diferentes, algo que nenhuma função consegue fazer.

Limitações que você precisa conhecer

O SWISH roda em servidor compartilhado e aplica um sandbox: recusa código que não consegue provar seguro, não mantém estado entre consultas e tem suporte limitado a entrada/saída e a concorrência. Para o laboratório da Aula 13 com CLP(FD) e para o Trabalho 5, prefira o SWI-Prolog instalado. O arquivo foi escrito sem acentos de propósito, para evitar avisos de codificação em ambientes com locale diferente.

Alternativas: OneCompiler · Prolog, JDoodle · Prolog, tutorialspoint.

JavaScript — console do navegador

Arquivo: aula01.js · Endereço: F12 em qualquer aba, ou playcode.io

Passo a passo

1. Pressione F12 em qualquer página e vá para a aba Console.
2. Cole o conteúdo de aula01.js e pressione Enter.
3. Se o navegador pedir, digite “allow pasting” antes de colar.

O que o programa mostra

Imperativo, funcional, classe do ES6 e o mesmo objeto feito por delegação de protótipos — o modelo real da linguagem, discutido na Aula 11.

Limitações que você precisa conhecer

O console mede tempo com `performance.now()`, que os navegadores arredondam por questão de segurança; espere variação entre execuções. Node.js em qualquer compilador online também roda o arquivo sem alteração.

Alternativas: playcode.io, glot.io/new/javascript, OnlineGDB (Node), Replit.

Java 17+ — OneCompiler

Arquivo: Aula01.java · Endereço: onecompiler.com/java

Passo a passo

1. Abra o endereço e cole o conteúdo de Aula01.java.
2. Confirme que o nome do arquivo no editor é Aula01.java — o nome precisa bater com a classe.
3. Clique em Run.

O que o programa mostra

Imperativo, streams, orientado a objetos e paralelo com `parallelStream` — quatro paradigmas dentro de uma linguagem que nasceu com um só.

Limitações que você precisa conhecer

Exige Java 17 ou superior por causa do `switch` com expressão e do `String.repeat`. Se a plataforma oferecer versão mais antiga, troque o `switch` por `if/else`. `parallelStream` usa o pool comum da JVM: em sandbox com um núcleo, ele não acelera nada — e esse é justamente um bom assunto para a Aula 14.

Alternativas: JDoodle, Replit, OnlineGDB, glot.io/new/java.

Plataformas multi-linguagem

Quando o playground oficial falhar, ou quando você quiser comparar duas linguagens na mesma tela.

Plataforma	Endereço	Cobre	Observação
glot.io	glot.io	44 linguagens, entre elas Elixir, Erlang, Haskell, Go, Java, JavaScript, Python, Clojure	Sem cadastro para executar. Não tem Prolog nem Racket.
OneCompiler	onecompiler.com	Racket, Prolog, Elixir, Java, Python, JavaScript e dezenas de outras	A escolha para Racket e Java. Confira a versão da linguagem antes de culpar seu código.
JDoodle	jdoodle.com	Racket, Prolog, Elixir, Java, C, Python e outras	Boa reserva quando o OneCompiler estiver lento.
OnlineGDB	onlinegdb.com	Python, Java, C, C++, JavaScript, Haskell, Go	Tem depurador passo a passo, útil na Aula 7 para observar estado e memória.
Ideone	ideone.com	Mais de 40 linguagens	Interface antiga, mas estável.
Replit	replit.com	Praticamente todas, com terminal e vários arquivos	Exige conta e o plano gratuito muda com frequência. Útil para o projeto do interpretador quando não houver máquina disponível.
Google Colab	colab.research.google.com	Python com bibliotecas científicas	Único da lista que produz gráficos de verdade. É a plataforma padrão dos 42 notebooks da disciplina.

Mapa: qual plataforma abrir em cada encontro

Aula	Linguagens	Onde rodar
1	Todas as oito	Uma aba por linguagem — o laboratório é justamente verificar o ambiente
2	Python	Colab: tokenizador, parser e AST desenhada em grafo
3	Python, Racket, JavaScript	Colab + OneCompiler: closures nas três linguagens
4	Python, Haskell	Colab + play.haskell.org: refatoração para estilo puro
5	Haskell ou Elixir	play.haskell.org ou glot.io: ADTs e pattern matching
6	Racket ou Python	OneCompiler ou Colab — o projeto do interpretador começa aqui
7	Python	Colab: aliasing, estado mutável e coleta de lixo
8	Python, TypeScript, Java, Haskell	Colab, playcode.io, OneCompiler, play.haskell.org
11	Python, JavaScript, Java	Colab + console do navegador + OneCompiler
12	SWI-Prolog	SWISH: base de conhecimento e consultas em várias direções
13	SWI-Prolog	SWI-Prolog local — o SWISH limita CLP(FD) e programas maiores
14	Python, Java	Colab e OneCompiler: condições de corrida e locks
15	Elixir e Go	glot.io e go.dev/play — mas o Trabalho 6 pede instalação local
16	Racket, Python	OneCompiler para macros; Colab para NumPy e JAX

Cinco limites que causam quase todos os problemas

Nenhum deles é defeito do seu código.

Relógio falso no Go Playground

O sandbox congela o tempo durante a computação. Medições de desempenho saem zeradas. Contorno: rodar localmente, ou aceitar a comparação apenas conceitual.

Sandbox do SWISH

Recusa código que não consegue provar seguro, não guarda estado entre consultas e restringe entrada/saída e concorrência. Contorno: SWI-Prolog instalado a partir da Aula 13.

Nenhuma persistência

Fechar a aba costuma apagar tudo. Nenhuma dessas plataformas é sistema de backup. Contorno: manter os arquivos no seu repositório Git e colar quando precisar.

Versões diferentes da linguagem

Uma mesma plataforma pode oferecer Elixir 1.9 e outra 1.16; Java 8 e Java 21. Contorno: conferir a versão antes de depurar um erro que talvez nem seja seu.

Limite de tempo e de memória

Quase todo compilador online corta a execução em poucos segundos. Nossos exemplos usam 200 mil elementos e cabem folgadoamente; um exercício mais pesado pode não caber. Contorno: reduzir o tamanho da entrada, ou rodar localmente.

Checklist do laboratório da Aula 1

Trabalhe em duplas. Marque cada linha quando a saída terminar com AMBIENTE OK. Ao final, envie um print da tela com as oito verificações.

OK?	Linguagem	Arquivo	Plataforma	Saída esperada termina com
☐	Python	aula01.py	Colab	AMBIENTE PYTHON OK
☐	Racket	aula01.rkt	OneCompiler	AMBIENTE RACKET OK
☐	Haskell	aula01.hs	play.haskell.org	AMBIENTE HASKELL OK
☐	Elixir	aula01.exs	glot.io	AMBIENTE ELIXIR OK
☐	Go	aula01.go	go.dev/play	AMBIENTE GO OK
☐	Prolog	aula01.pl	SWISH	AMBIENTE SWI-PROLOG OK
☐	JavaScript	aula01.js	Console F12	AMBIENTE JAVASCRIPT OK
☐	Java	Aula01.java	OneCompiler	AMBIENTE JAVA OK

O que observar enquanto testa

As oito versões resolvem o mesmo problema e imprimem o mesmo número. O que muda é o caminho: onde há mutação de estado e onde não há; onde a concorrência aparece como canal, como processo isolado ou como lock; onde o objeto precisa de uma palavra-chave e onde ele é apenas uma closure. Anote uma diferença que tenha surpreendido você — ela vale meio ponto na discussão do fim da aula.

Conferência das informações

Endereços e limitações conferidos em julho de 2026 junto às fontes oficiais: página do Haskell Playground e o anúncio do comitê Haskell.org; documentação de introdução do SWI-Prolog, que descreve as restrições do sandbox do SWISH; páginas de linguagem do OneCompiler, do JDoodle e do glot.io; e o site oficial do Elixir quanto ao Livebook. Plataformas online mudam de política com frequência — se algum endereço falhar, use a coluna de alternativas e avise o professor para atualização deste guia.